



Implementing Generative Adversarial Networks

Guest Lecture | Winter 2026

Generative vs. Discriminative Models

Discriminative Models

Standard classifiers (e.g., Is this a cat or dog?). They model the decision boundary between classes.

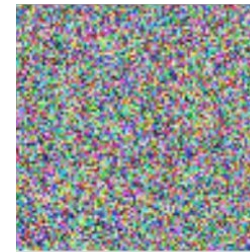
$$P(Y | X)$$

Generative Models

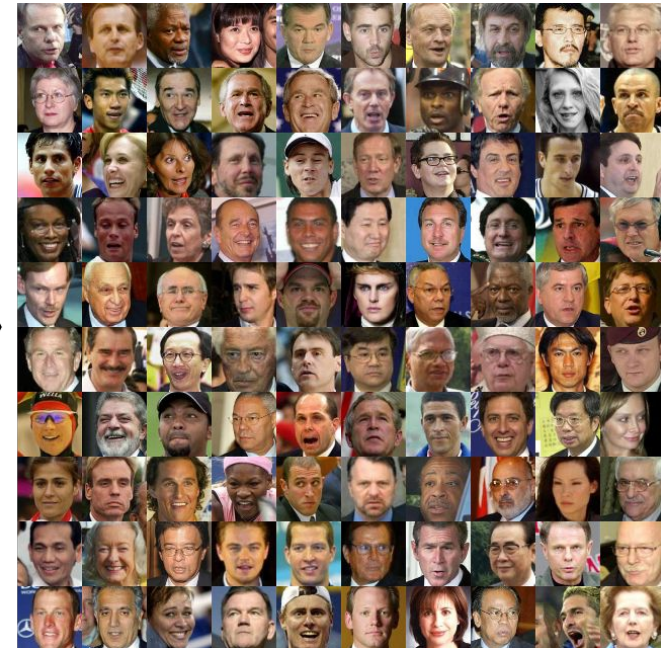
They model the distribution of the data itself. They can generate **new** samples that look like the original data.

$$P(X)$$

Noise $\sim N(0,1)$



Generative
Model



Why do we care about GANs?



Creative Content

Generating art, music, and realistic textures for video games (e.g., NVIDIA Canvas).



Super Resolution

Upscaling low-quality images to high definition by "hallucinating" plausible details.



Synthetic Data

Creating medical training data where real patient data is scarce or privacy-restricted.

Visual Motivation: Super Resolution

Traditional resizing methods (like bicubic interpolation) result in blurry images.

GANs learn the **texture** of the real world, allowing them to fill in details like eyelashes, hair strands, and skin pores that didn't exist in the input.

This is possible because the Discriminator punishes "blurry" results.



The Core Intuition

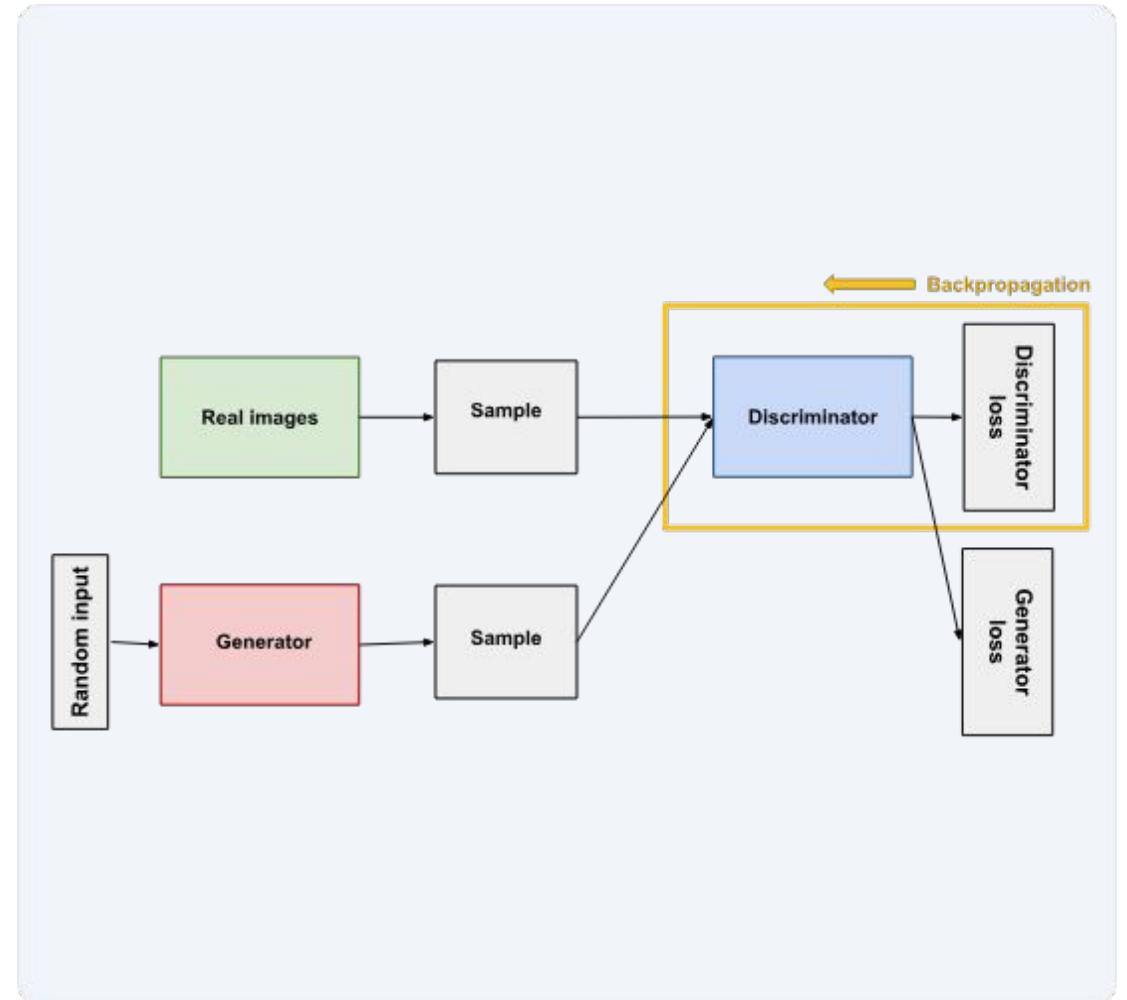
The Art Forger (Generator)

Starts by making random scribbles. Wants to sell them as authentic masterpieces.

The Art Detective (Discriminator)

Examines real art (Training Data) and the Forger's work. Tries to classify them as "Real" or "Fake".

The Game: The Detective provides feedback ("This looks fake because..."). The Forger learns and improves. Eventually, the Detective can't tell the difference.



Part 1: The Math

Formalizing the Adversarial
Game

Defining the Players

Generator (G)

- ✓ **Input:** A noise vector sampled from usually Gaussian space.
- ✓ **Output:** A fake sample `x_fake = G(z)`
- ✓ **Goal:** Fool D into outputting 1.

Discriminator (D)

- ✓ **Input:** Data `x` (Real or Fake).
- ✓ **Output:** Probability `D(x) ∈ [0, 1]`
- ✓ **Goal:** Output 1 for Real, 0 for Fake.

The Minimax Game

We train both networks simultaneously using a single value function $V(D, G)$

$$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$$

Term 1 (Real): $\log D(x)$ D wants to maximize this (push $D(x)$ to 1).

Term 2 (Fake): $\log(1 - D(G(z)))$ D wants to maximize this (push $D(G(z))$ to 0). G wants to minimize this.

Implementation Detail: The "Log Trick"

The Problem

Ideally, G wants to minimize. $\log(1 - D(G(z)))$

However, early in training, D is very good and G is terrible. $D(G(z)) \approx 0$

The gradient of this function at 0 is nearly flat. G stops learning.

The Solution

Instead of minimizing the likelihood of being caught, we **maximize** the likelihood of fooling D.

Maximize: $\log(D(G(z)))$

This provides strong gradients even when the Generator is poor.



Part 2: Implementation Setup

Data, Architecture, and
Hyperparameters

Preprocessing is Critical

1. Normalization

We use the `Tanh` activation function in the Generator, which outputs values in range `[-1, 1]`.

Therefore, we MUST normalize our input images to this same range.

2. Flattening (For FC Networks)

If using simple Linear layers (not CNNs), we must flatten the 28×28 MNIST image into a vector of 784.

```
import torchvision.transforms as transforms

# Define Transform
transform = transforms.Compose([
    transforms.ToTensor(),
    # (image - 0.5) / 0.5 → [-1, 1]
    transforms.Normalize((0.5,), (0.5,))
])

# Load Data
dataloader = DataLoader(
    MNIST('.', transform=transform, download=True),
    batch_size=64, shuffle=True
)
```

The Generator (PyTorch)

Input: Latent Vector z (size 64).

Hidden: Linear Layers + **ReLU**.

Output: Linear Layer + **Tanh**.

Pro Tip: Batch Normalization is often added after linear layers in deeper networks (like DCGAN) to stabilize training.

```
class Generator(nn.Module):
    def __init__(self, z_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(z_dim, 256),
            nn.ReLU(True),
            nn.Linear(256, 784),
            nn.Tanh() # Output -1 to 1
        )

    def forward(self, z):
        return self.net(z)
```

The Discriminator (PyTorch)

Input: Image Vector (size 784).

Hidden: Linear Layers + **LeakyReLU**.

Output: Linear Layer + **Sigmoid** (Probability).

Why LeakyReLU? Standard ReLU kills gradients for negative inputs. In GANs, it's crucial that gradients flow even when the Discriminator rejects an image, so G can learn *why* it was rejected.

```
class Discriminator(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.Linear(784, 256),  
            nn.LeakyReLU(0.2),  
            nn.Linear(256, 1),  
            nn.Sigmoid() # Output 0 to 1  
        )  
  
    def forward(self, x):  
        return self.net(x)
```

Part 3: The Training Loop

The Heart of the
Implementation

Anatomy of a Single Epoch

01

Prepare Data

Sample a batch of real images from the dataloader.
Sample a batch of noise vectors .

02

Train Discriminator

Calculate loss on Real vs Fake. Update **only** D's weights. (Detach G).

03

Train Generator

Generate Fake images.
Calculate loss based on D's feedback. Update **only** G's weights.

Step 1: Train Discriminator

Goal: Correctly identify Real (1) and Fake (0).

```
# 1. Train on Real Data
real_preds = D(real_images)
d_loss_real = criterion(real_preds, torch.ones_like(real_preds))

# 2. Train on Fake Data
noise = torch.randn(batch_size, z_dim)
fake_images = G(noise)

# IMPORTANT: .detach() prevents gradients flowing to G
fake_preds = D(fake_images.detach())
d_loss_fake = criterion(fake_preds, torch.zeros_like(fake_preds))

# 3. Update Weights
d_loss = d_loss_real + d_loss_fake
d_optimizer.zero_grad()
d_loss.backward()
d_optimizer.step()
```

Step 2: Train Generator

Goal: Fool the Discriminator (make D output 1).

```
# 1. Generate (or reuse) Fake Images
# No .detach() here! We need gradients to flow through D to G
fake_preds_for_g = D(fake_images)

# 2. Calculate Loss with FLIPPED Labels
# We want D to predict 1 (Real) for these fakes
g_loss = criterion(fake_preds_for_g, torch.ones_like(fake_preds_for_g))

# 3. Update Weights
g_optimizer.zero_grad()
g_loss.backward()
g_optimizer.step()
```

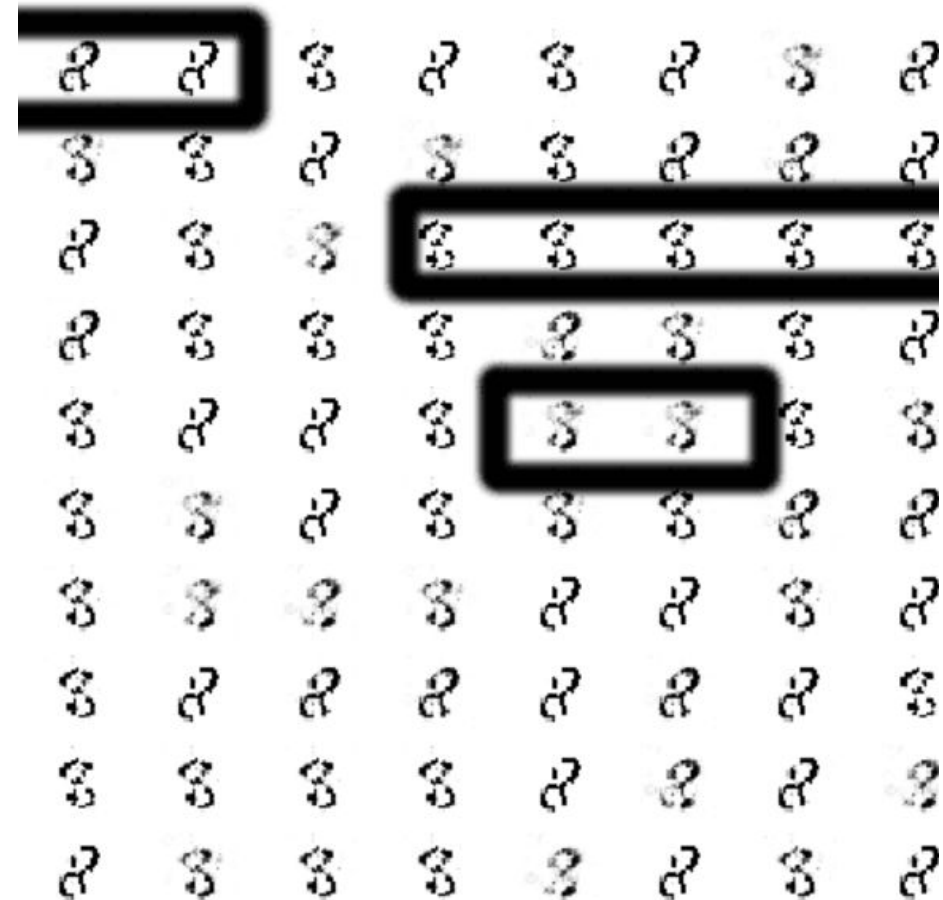
Note: We effectively treat D as a "frozen" loss function during this step.

Common Failure: Mode Collapse

Sometimes, the Generator finds a single output that successfully fools the Discriminator.

Instead of learning the full distribution (all digits 0-9), it maps **every** noise vector to that single "safe" image (e.g., only generates 1s).

Fixes: Unrolled GANs, Mini-batch discrimination, or Wasserstein GAN (WGAN).



Visualizing Success: Interpolation

How do we know G isn't just memorizing the training data?

We take two random noise vectors, z_1 and z_2 , and linearly interpolate between them.

If the model has learned the data **manifold**, the generated images should morph smoothly from one to the other.

Interpolation



GANSpace edits



Evaluating GANs

Inception Score (IS)

Measures image clarity and diversity. Higher is better.

Fréchet Inception Distance (FID)

Measures statistical distance between real and fake distributions. Lower is better. This is the industry standard.



Part 4: Scaling Up to DCGAN

Deep Convolutional GANs

Comparison: Vanilla vs. DCGAN

Vanilla GAN

- ✓ **Layers:** Fully Connected (MLP).
- ✓ **Input:** Flattened Vectors (1D).
- ✓ **Limitation:** Destroys spatial structure. Huge parameter count for large images.
- ✓ **Use Case:** Simple data (MNIST).

DCGAN

- ✓ **Layers:** Convolutional (CNN).
- ✓ **Input:** 3D Tensors (C, H, W).
- ✓ **Advantage:** Preserves spatial features.
Parameter efficient via weight sharing.
- ✓ **Use Case:** Realistic Images (Faces, Scenes).

Implementing DCGAN Generator

We use ConvTranspose2d to upsample.

```
# Input: z (Batch, 100, 1, 1) → Output: Image (Batch, 3, 64, 64)
nn.Sequential(
  # Block 1: 1x1 → 4x4
  nn.ConvTranspose2d(100, 512, 4, 1, 0, bias=False),
  nn.BatchNorm2d(512),
  nn.ReLU(True),
  # Block 2: 4x4 → 8x8
  nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False),
  nn.BatchNorm2d(256),
  nn.ReLU(True),
  # ... (Repeat to reach 64x64)
)
```

Implementing DCGAN Discriminator

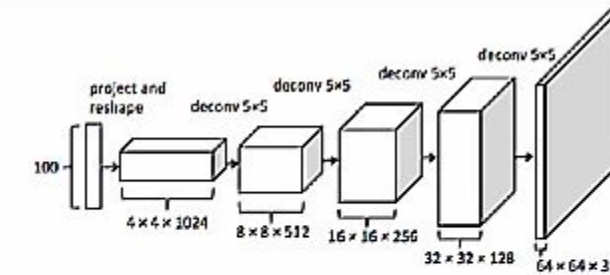
We use Conv2d with stride=2 to downsample.

```
# Input: Image (Batch, 3, 64, 64) → Output: Probability
nn.Sequential(
    # Block 1: 64x64 → 32x32
    nn.Conv2d(3, 64, 4, 2, 1, bias=False),
    nn.LeakyReLU(0.2, inplace=True),
    # Block 2: 32x32 → 16x16
    nn.Conv2d(64, 128, 4, 2, 1, bias=False),
    nn.BatchNorm2d(128),
    nn.LeakyReLU(0.2, inplace=True),
    # ... (Repeat to reach 1x1)
)
```

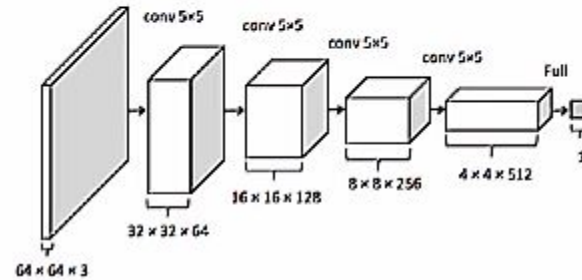
DCGAN Architecture

DCGAN Overall

Generator



Discriminator



The generator takes a 100-dim vector and projects it through a series of transposed convolutions to create a 64x64 image.

Crucial Step: Weight Initialization

DCGANs will **not converge** with default PyTorch initialization.

We must initialize all weights from a Normal distribution with mean=0, std=0.02.

```
def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

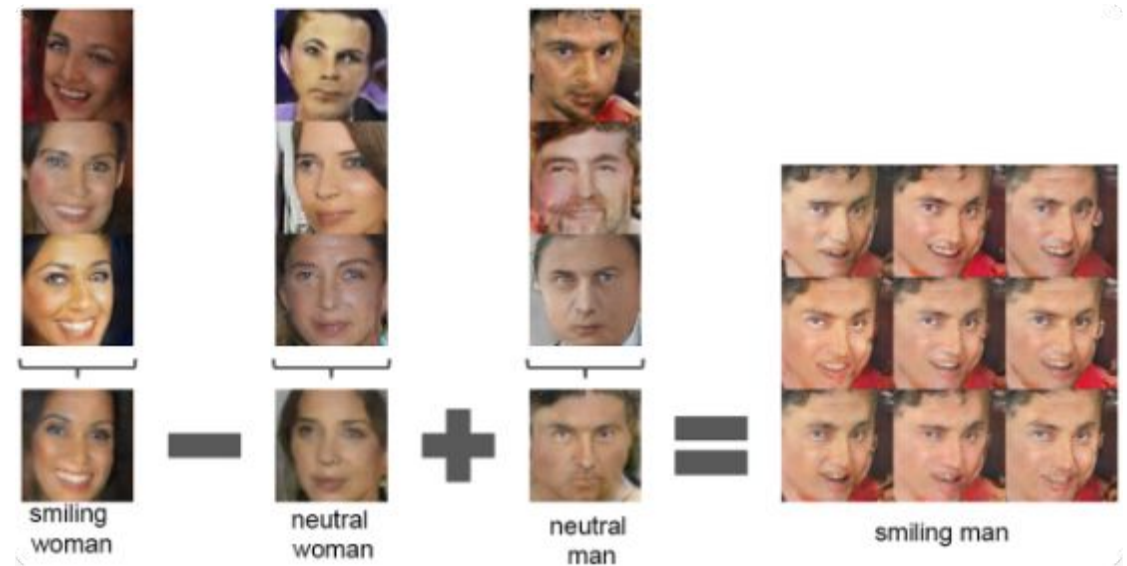
# Apply it!
netG.apply(weights_init)
netD.apply(weights_init)
```

Results: CelebA Faces

From Digits to Faces

While Vanilla GANs struggle with anything more complex than MNIST, DCGANs can generate plausible human faces.

Training time: ~2 hours on a modern GPU.



Generated samples after 5 epochs.

Summary

`</>`Implementation

Two networks, two optimizers, two loss calculations per step. Watch out for normalization!

Stability

GANs are unstable. Loss oscillation is normal. Use heuristics (LeakyReLU, soft labels) to stabilize.

Questions?

Synthetic Supervision: Leveraging Generative Models

Generative AI Enables Medical Image Segmentation in Ultra Low-Data Regimes
(Nature Communications)

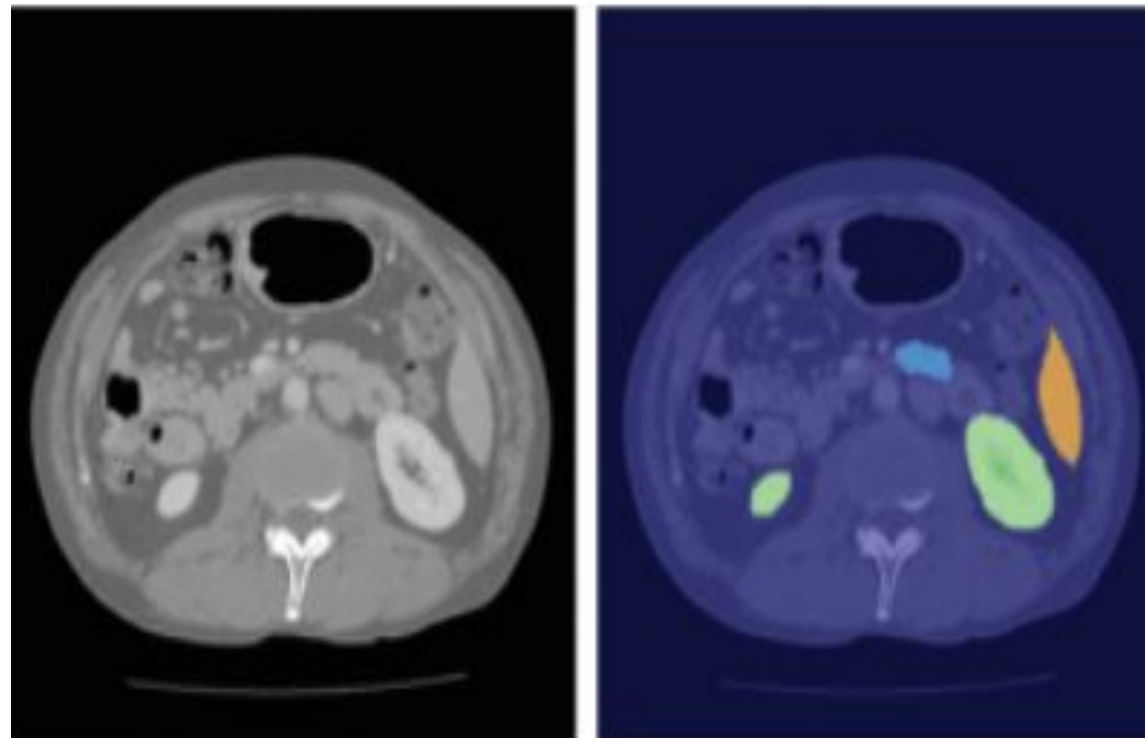
Outline

1. Motivation
2. Proposed Method
3. Experimental Analysis
4. Summary

□ Motivation

Medical Image Semantic Segmentation

- Predict the semantic category of **every pixel** in a medical image.
- Broad applications in disease detection, organ recognition, tissue measurement, etc.



□ Motivation

Deep Learning for Semantic Segmentation

- Strong performance: U-Net, DeepLab, SegFormer, etc.
- Limitations: **Data hungry**. Training these models requires **large amounts of precisely labeled** segments.



Semantic masks of medical images are difficult to obtain

- Objects have complex structures
- Require strong domain expertise
- Privacy concern
- ...

□ Motivation

Data Augmentation Methods

- Commonly used methods
 - Basic image transformations: flipping, rotation, translation, etc.
 - Generative model-based methods: DatasetGAN, WGAN, etc.

Data augmentation and segmentation model training are performed separately. Augmented data may not be suitable for improving the performance of segmentation model.

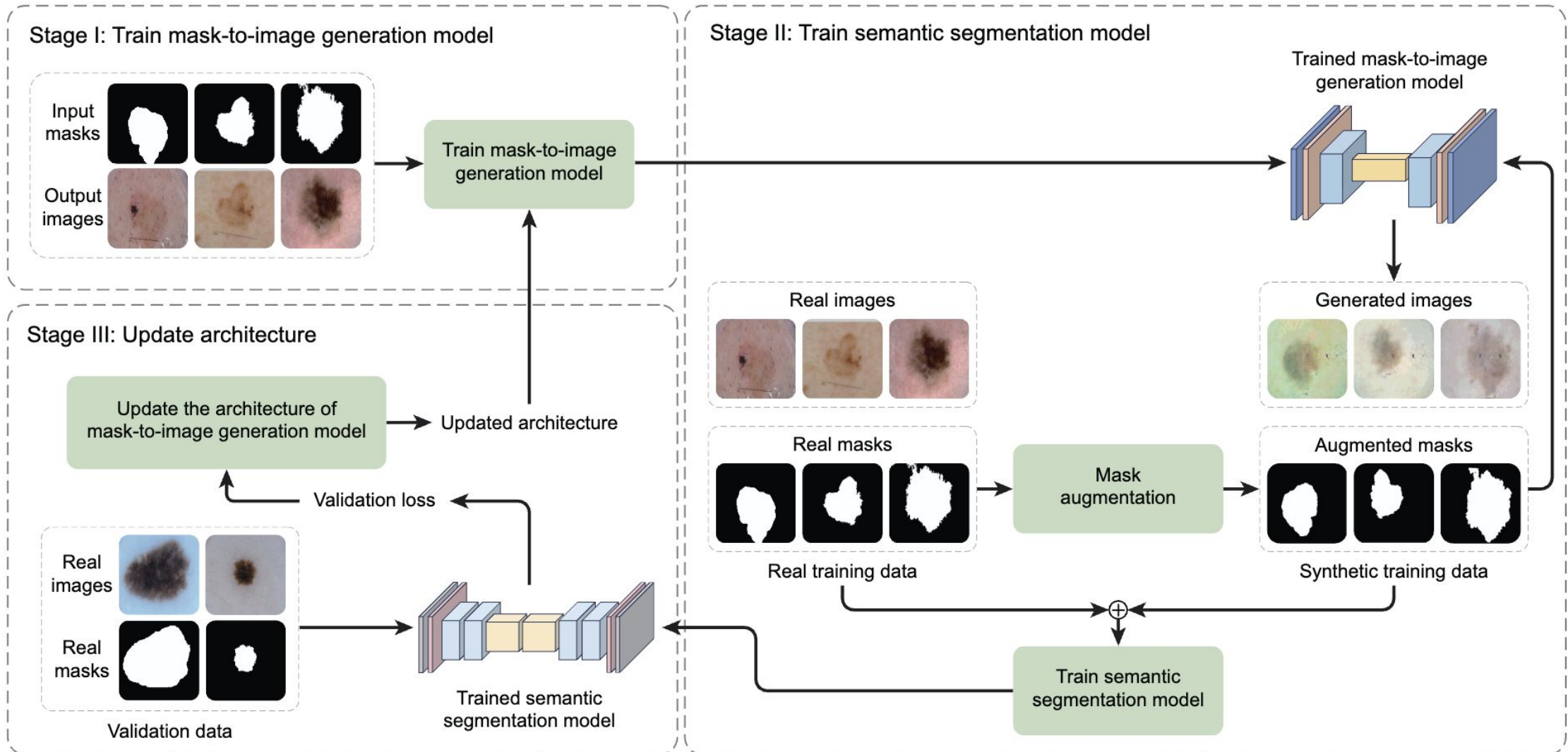
Semi-supervised Learning in Semantic Segmentation

- Commonly used methods: Deep co-training, Cross teaching between CNN and transformer, etc.

In medical domains, even unlabeled data are challenging to obtain due to data privacy concerns, regulation barriers, etc.

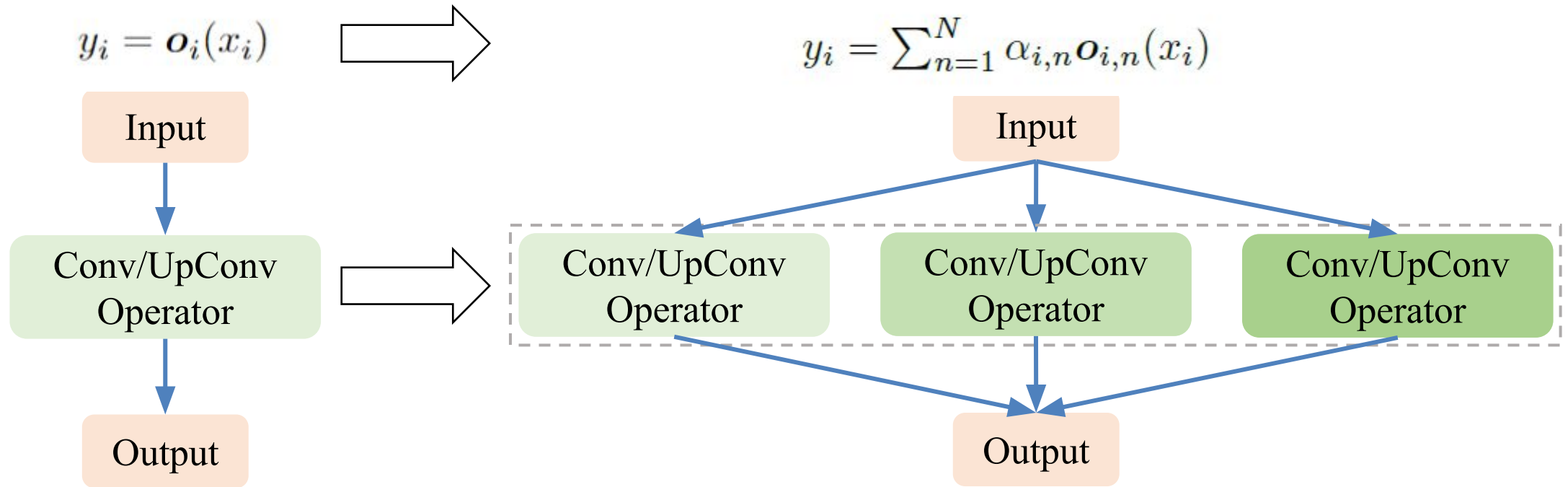
Method: Overview

GenSeg: an end-to-end data generation framework designed to generate high-quality, labeled data, to enable the training of accurate medical image segmentation models in ultra low-data regimes

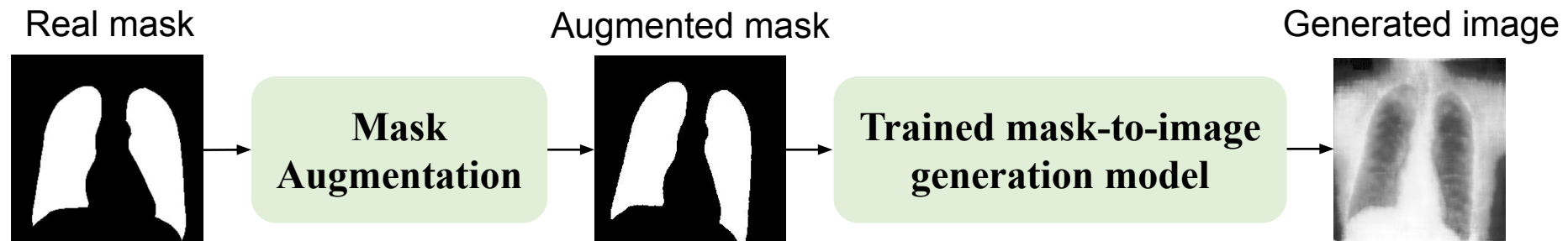


□ Method: Details

How to make the architecture of augmentation model to be differentiable



How to generate synthetic image-mask pairs



□ Method: Optimization problems

Stage I

$$G^*(A) = \arg \min_G \max_D \mathcal{L}_{gan}(D_{gan}; G, D, A)$$

Diagram illustrating the components of the GAN loss function $\mathcal{L}_{gan}$:

- Generator (points to G)
- Discriminator (points to D)
- Architecture (points to A)

Stage II

$$W^*(G^*(A), A) = \arg \min_W \mathcal{L}_{seg}(D_{tr}; W) + \lambda \mathcal{L}_{seg}(D_{aug}(D_{tr}; G^*(A), A); W)$$

Diagram illustrating the components of the segmentation loss function $\mathcal{L}_{seg}$:

- Segmentation model (points to W)

Stage III

$$\min_A \mathcal{L}_{seg}(D_{val}; W^*(G^*(A), A))$$

A multi-level optimization-based framework

$$\begin{aligned} \min_A \quad & \mathcal{L}_{seg}(D_{val}; W^*(G^*(A), A)) \\ s.t \quad & W^*(G^*(A), A) = \arg \min_W \mathcal{L}_{seg}(D_{tr}; W) + \lambda \mathcal{L}_{seg}(D_{aug}(D_{tr}; G^*(A), A); W) \\ & G^*(A), E^* = \arg \min_G \max_E \mathcal{L}_{gan}(D_{gan}; G, A, E) \end{aligned}$$

Experimental results

In-domain performance

Experimental settings

- Training and testing data are from the same datasets.

Tasks

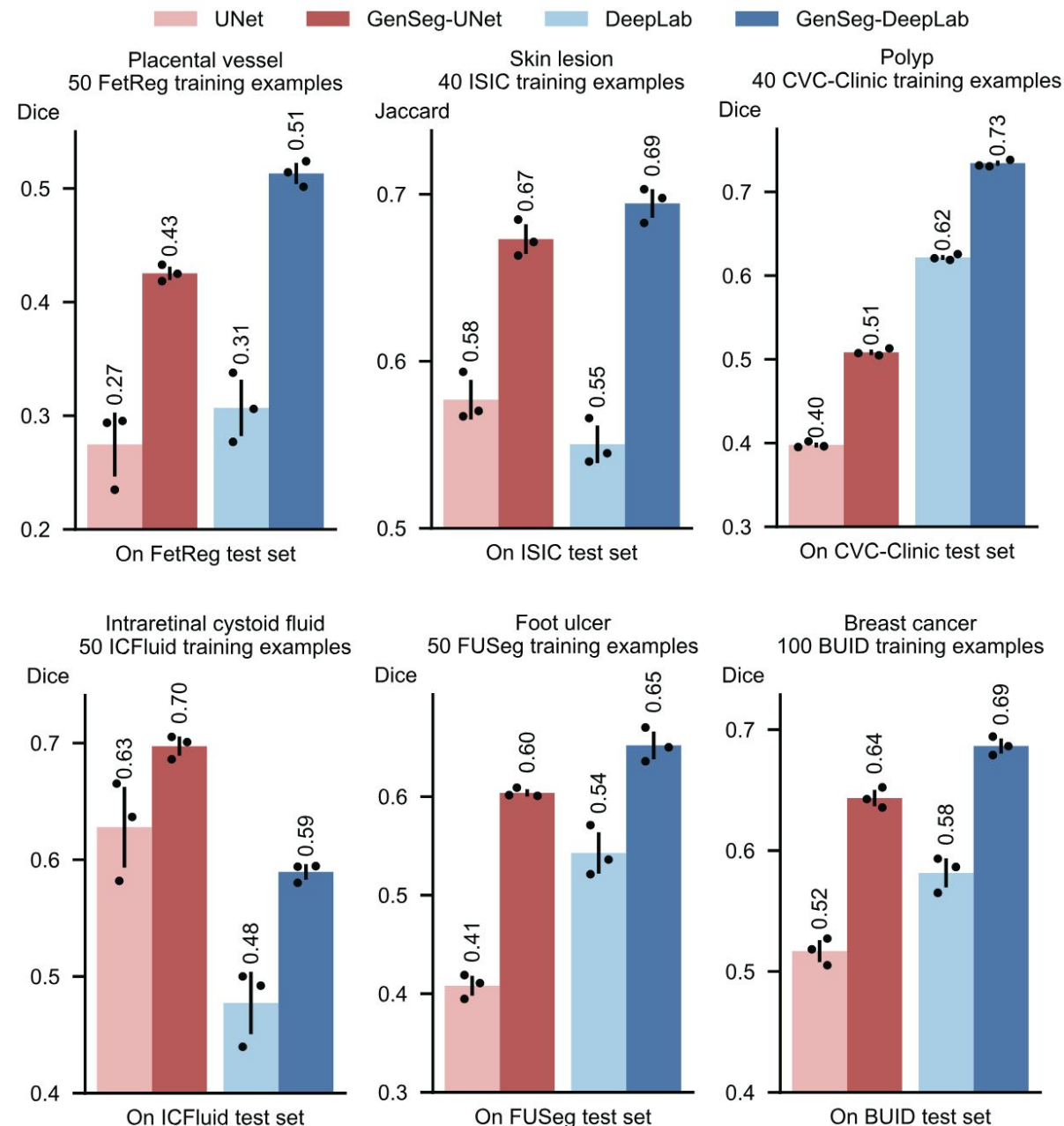
- Placental vessels, skin lesions, polyps, intraretinal cystoid fluids, foot ulcers, and breast cancer segmentations.

Baselines

- UNet and DeepLab

Findings

- After applying GenSeg to UNet and DeepLab, models' performance can be improved greatly for in-domain evaluation.



Experimental results

Out-of-domain performance

Experimental settings

- Training and testing data are from the different datasets.
- Models are trained on one dataset (e.g. ISIC), and evaluated on other datasets (e.g. DermIS and PH2) directly.

Tasks

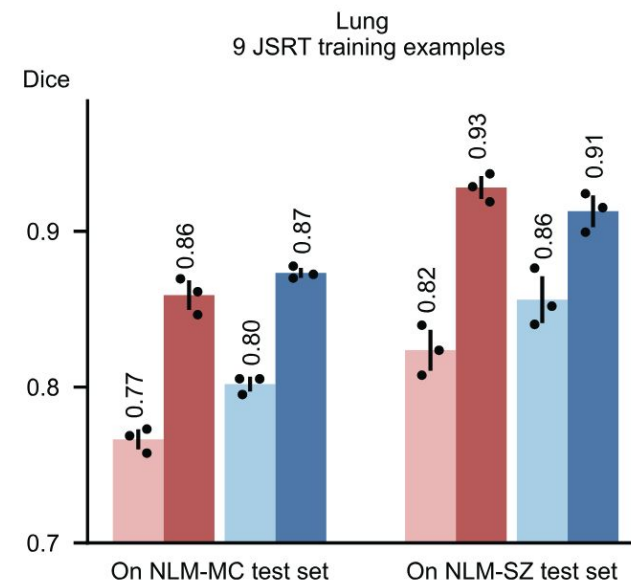
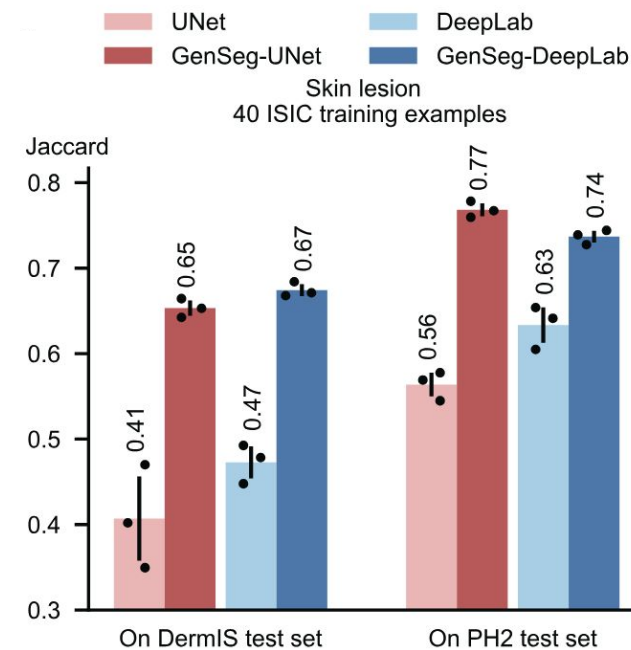
- Skin lesions and lungs segmentation.

Baselines

- UNet and DeepLab

Findings

- After applying GenSeg to UNet and DeepLab, models' performance can be improved greatly for out-of-domain evaluation.



Experimental results

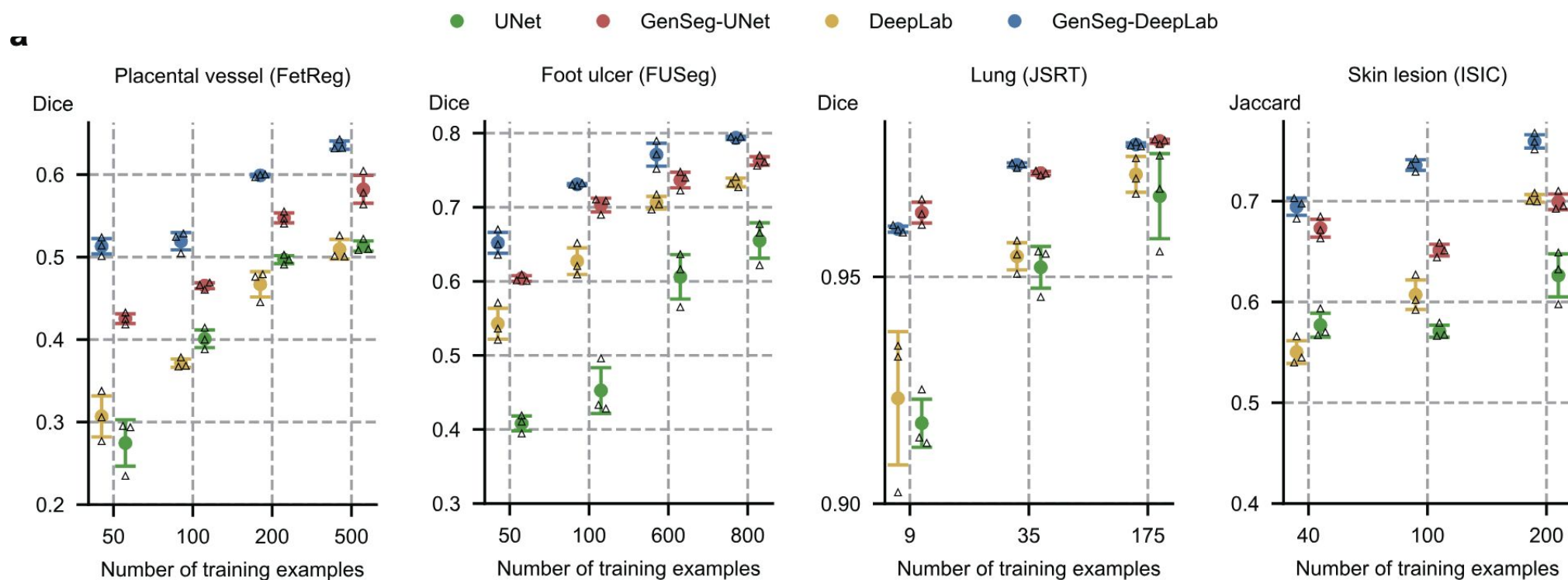
Data efficiency

Experimental settings

- Evaluate models' performance with different numbers of training examples.

Findings

- GenSeg achieves comparable performance to baselines with significantly fewer training examples.



□ Experimental results

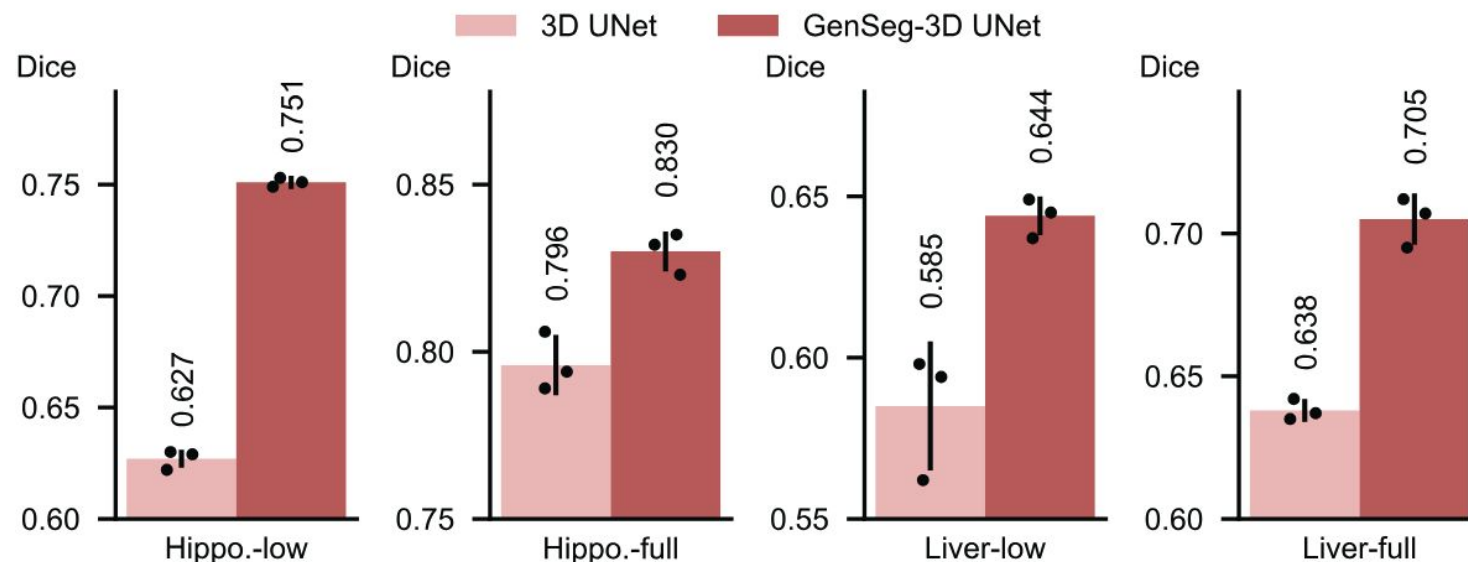
Extension for 3D segmentation tasks

□ Experimental settings

- we adapted GenSeg by using 3D Unet as the segmentation model and Pix2PixNIfTI as the generative model. Experiments are conducted on hippocampus and liver segmentation tasks from MSD challenge.

□ Findings

- GenSeg consistently improved segmentation performance over the baseline 3D UNet in ultra-low data settings (40 training volumes) and higher data settings using the full available training sets (208 volumes for hippocampus and 98 for liver).

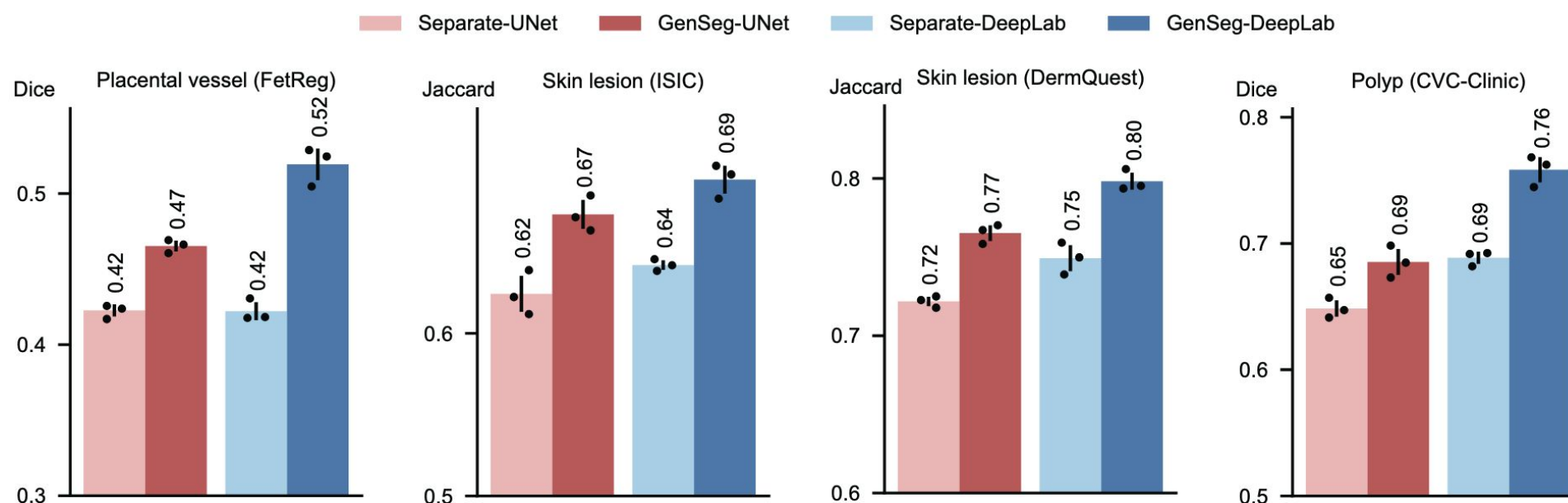


Experimental results

Ablation studies

GenSeg's end-to-end generation mechanism

- We compared with a separate baseline, which separates data generation from segmentation model training.
- In Separate, the mask-to-image generation model is initially trained and then fixed. Subsequently, it generates data, which is then utilized to train the segmentation model.

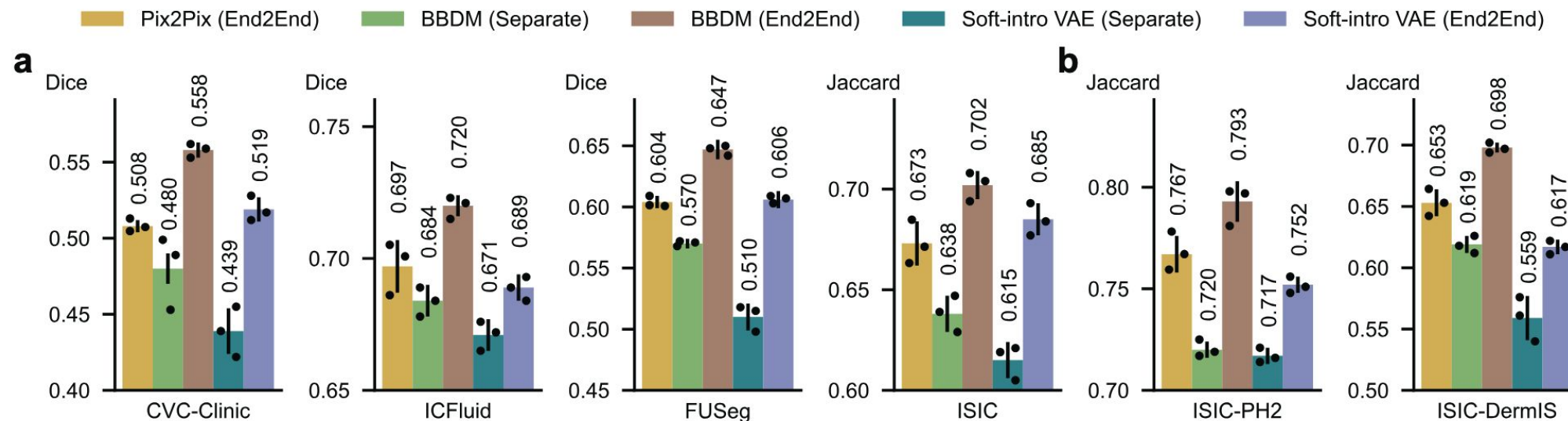


Experimental results

Ablation studies

□ Different mask-to-image generative models

- We applied Pix2Pix (GAN-based) model as the default generative model
- We compared with BBDM (Diffusion-based) and Soft-intro VAE (VAE-based) models to investigate how different choices of mask-to-image generative models affect the final segmentation performance.



□ Summary

□ Method

- We propose GenSeg, a generative deep learning framework designed to address the challenges of ultra low-data regimes in medical image segmentation.
- GenSeg performs end-to-end data generation tightly coupled with segmentation objectives

□ Experiment

- We validated GenSeg across various segmentation tasks.